Fast Fourier Transform

1. Why This Matters

When we want to multiply two polynomials or signals, we need convolution.
Example:

```
A(x) = 1 + 2x + 3x²
B(x) = 1 + 0x + 1x²
```

Multiplying these polynomials gives new coefficients (that's convolution).

Naive way: For every term in A, multiply with every term in B → O(n²).
Fast way: Use **FFT (Fast Fourier Transform)** with divide & conquer 2. Key Idea of FFT

The Discrete Fourier Transform (DFT) takes a sequence (like [1,2,3,4]) and rewrites it in terms of frequencies.→ O(n log n).

2. Key Idea of FFT

The **Discrete Fourier Transform (DFT)** takes a sequence (like `[1,2,3,4]`) and rewrites it in terms of frequencies.

The magic is:

- In frequency space, *convolution becomes simple multiplication*.

So:

1. FFT(A), FFT(B).
2. Multiply them point by point.
3. Inverse FFT to go back to time/polynomial space.

That gives us the convolution result.

3. Divide & Conquer Structure

FFT works because of symmetry in complex "roots of unity".

You don't need to derive formulas — just follow the pattern:

1. **Divide**
   Split array into evens and odds.
   Example: `[a0, a1, a2, a3]` → Evens = `[a0, a2]`, Odds = `[a1, a3]`.
2. **Conquer**

Recursively compute FFT on evens and odds (smaller problems).

3. **Combine**
   For each k:

```
A[k]       = E[k] + w^k * O[k]
A[k+n/2]   = E[k] - w^k * O[k]
```

   where `w` is the complex n-th root of unity:
   `cos(2πk/n) + i*sin(2πk/n)`

   That's it. The recursion stops when `n=1`.

4. Inverse FFT

To come back to original space:

- Do the same recursion, but use `w^-1` (conjugates of roots of unity).
- After recursion, divide every term by n.

5. What You Need to Code

You need two main parts:

```cpp
// Recursive FFT
void fft(vector<complex<double>>& a, bool invert);

// Convolution using FFT
vector<long long> convolution(const vector<int>& A, const vector<int>& B);
```

Steps in `convolution()`:

1. Pad A and B with zeros so their size = next power of 2 ≥ (n+m).
2. FFT both sequences.
3. Multiply results element-wise.
4. Inverse FFT the product.
5. Round real parts to nearest integers.

6. Input/Output Format

Input

```
n m
```

```
A0 A1 ... A(n-1)
B0 B1 ... B(m-1)
```

Output

```
C0 C1 ... C(n+m-2)
```

Example

Input

```
4 4
1 2 3 4
1 0 1 0
```

Output

```
1 2 4 6 3 4 0
```

7. Hints to Students

- Use `std::complex<double>` (already in `<complex>`).
- Always check base case: if `n == 1`, return.
- Inverse FFT: set `invert = true` and divide by `n`.
- Don't forget to round at the very end.

8. References (Videos / Visual Intuition)

If the recursion feels too abstract, watch one of these:

- MIT OCW – Fast Fourier Transform explained (math + code intuition).
- Reducible's Visual FFT Explanation (intuitive animation, no heavy math).

Watch one video + read this sheet → you'll be ready to code.

👉 **Your Task:** Implement convolution using FFT with divide & conquer.